

2906. Construct Product Matrix

Solved 

Medium

 Topics

 Companies

 Hint

Given a **0-indexed** 2D integer matrix `grid` of size `n * m`, we define a **0-indexed** 2D matrix `p` of size `n * m` as the **product** matrix of `grid` if the following condition is met:

- Each element `p[i][j]` is calculated as the product of all elements in `grid` except for the element `grid[i][j]`. This product is then taken modulo `12345`.

Return *the product matrix of* `grid`.

Example 1:

Input: `grid = [[1,2],[3,4]]`

Output: `[[24,12],[8,6]]`

Explanation: $p[0][0] = \text{grid}[0][1] * \text{grid}[1][0] * \text{grid}[1][1] = 2 * 3 * 4 = 24$

$p[0][1] = \text{grid}[0][0] * \text{grid}[1][0] * \text{grid}[1][1] = 1 * 3 * 4 = 12$

$p[1][0] = \text{grid}[0][0] * \text{grid}[0][1] * \text{grid}[1][1] = 1 * 2 * 4 = 8$

$p[1][1] = \text{grid}[0][0] * \text{grid}[0][1] * \text{grid}[1][0] = 1 * 2 * 3 = 6$

So the answer is `[[24,12],[8,6]]`.

Example 2:

Input: `grid = [[12345],[2],[1]]`

Output: `[[2],[0],[0]]`

Explanation: $p[0][0] = \text{grid}[0][1] * \text{grid}[0][2] = 2 * 1 = 2$.

$p[0][1] = \text{grid}[0][0] * \text{grid}[0][2] = 12345 * 1 = 12345$. $12345 \% 12345 = 0$. So $p[0][1] = 0$.

$p[0][2] = \text{grid}[0][0] * \text{grid}[0][1] = 12345 * 2 = 24690$. $24690 \% 12345 = 0$. So $p[0][2] = 0$.

So the answer is `[[2],[0],[0]]`.

Constraints:

- $1 \leq n == \text{grid.length} \leq 10^5$
- $1 \leq m == \text{grid}[i].\text{length} \leq 10^5$
- $2 \leq n * m \leq 10^5$
- $1 \leq \text{grid}[i][j] \leq 10^9$

Python:

class Solution:

def constructProductMatrix(self, grid):

 m, n = len(grid), len(grid[0])

 arr = [grid[i][j] for i in range(m) for j in range(n)]

 size = len(arr)

 pre = [1]*size

 suf = [1]*size

 for i in range(1, size):

 pre[i] = (pre[i-1] * arr[i-1]) % 12345

 for i in range(size-2, -1, -1):

 suf[i] = (arr[i+1] * suf[i+1]) % 12345

 res = [(pre[i]*suf[i]) % 12345 for i in range(size)]

 ans = [[0]*n for _ in range(m)]

 idx = 0

 for i in range(m):

 for j in range(n):

```
    ans[i][j] = res[idx]
    idx += 1
```

```
    return ans
```

JavaScript:

```
var constructProductMatrix = function(grid) {
    let m = grid.length, n = grid[0].length;
    let arr = [];

    for(let i=0;i<m;i++){
        for(let j=0;j<n;j++){
            arr.push(grid[i][j]);
        }
    }

    let size = arr.length;
    let pre = new Array(size).fill(1);
    let suf = new Array(size).fill(1);

    for(let i=1;i<size;i++){
        pre[i] = (pre[i-1] * arr[i-1]) % 12345;
    }

    for(let i=size-2;i>=0;i--){
        suf[i] = (arr[i+1] * suf[i+1]) % 12345;
    }

    let res = new Array(size);
    for(let i=0;i<size;i++){
        res[i] = (pre[i] * suf[i]) % 12345;
    }

    let ans = Array.from({length:m},()=>Array(n).fill(0));
    let idx = 0;
    for(let i=0;i<m;i++){
        for(let j=0;j<n;j++){
            ans[i][j] = res[idx++];
        }
    }

    return ans;
};
```

Java:

```
class Solution {
    public int[][] constructProductMatrix(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int size = m * n;
        int[] arr = new int[size];

        int idx = 0;
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                arr[idx++] = grid[i][j];
            }
        }

        int[] pre = new int[size];
        int[] suf = new int[size];
        Arrays.fill(pre,1);
        Arrays.fill(suf,1);

        for(int i=1;i<size;i++){
            pre[i] = (int)((1L * pre[i-1] * arr[i-1]) % 12345);
        }

        for(int i=size-2;i>=0;i--){
            suf[i] = (int)((1L * arr[i+1] * suf[i+1]) % 12345);
        }

        int[] res = new int[size];
        for(int i=0;i<size;i++){
            res[i] = (int)((1L * pre[i] * suf[i]) % 12345);
        }

        int[][] ans = new int[m][n];
        idx = 0;
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
                ans[i][j] = res[idx++];
            }
        }

        return ans;
    }
}
```